Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 12

по курсу «Методы оптимизации»

«Генетический алгоритм»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Реализовать генетический алгоритм

2 Результаты

Исходный код программы представлен в листинге 1.

```
from PIL import Image, ImageDraw, ImageChops
import numpy as np
import copy
import time
import pandas as pd
import os


def draw(polygons, size):
    """
                                        """
    img = Image.new('RGB', (size[0], size[1]), (255, 255, 255))
    drw = ImageDraw.Draw(img, 'RGBA')
    for pol in polygons:
        drw.polygon(pol['vertices'], pol['RGBA'])
    return img.convert("RGB")


def random_triangles(N, size, vertices=3, colour="random", alpha="random"):
    """
                                                        """
    collection = []
    for i in range(N):
        coords = [(np.random.randint(0, size[0]), np.random.randint(0, size[1]))
                  for _ in range(vertices)]
        if alpha == "random":
            a = np.random.randint(0, 50)
        else:
            a = alpha
        if colour == "white":
            rgba = (255, 255, 255, a)
        elif colour == "black":
            rgba = (0, 0, 0, a)
```

```
        elif colour == "random":
            rgba = (np.random.randint(0, 256), np.random.randint(0, 256),
                    np.random.randint(0, 256), a)
        triangle = {"vertices": coords, "RGBA": rgba}
        collection.append(triangle)
    return collection


def pixel_difference(candidate, target):
    """
                                    """
    diff = ImageChops.difference(candidate, target)
    return np.array(diff.getdata()).sum()


def max_pixel_difference(target):
    """
                                        """
    white = Image.new('RGB', target.size, (255, 255, 255))
    diff = ImageChops.difference(white, target)
    return np.array(diff.getdata()).sum()


def mutation_modify(mutant, size):
    """                    :
            (                                    ). """
    polidx = np.random.randint(len(mutant))
    pol = mutant[polidx]
    if np.random.random() < 0.5:
        vidx = np.random.randint(len(pol['vertices']))
        pol['vertices'][vidx] = (
            np.random.randint(0, size[0]),
            np.random.randint(0, size[1])
        )
    else:
        r, g, b = np.random.randint(0, 256, 3)
        a = np.random.randint(0, 256)
        pol['RGBA'] = (int(r), int(g), int(b), a)


def mutation_delete(mutant):
```

```
    """                    :
            (                              ). """
    if len(mutant) <= 1:
        return
    idx = np.random.randint(len(mutant))
    mutant.pop(idx)


def mutation_insert(mutant, size, num_verts=3):
    """                    :
            (                                        ). """
    coords = [(np.random.randint(-10, size[0] + 10),
               np.random.randint(-10, size[1] + 10)) for _ in range(num_verts)]
    rgba = (np.random.randint(0, 256), np.random.randint(0, 256),
            np.random.randint(0, 256), np.random.randint(0, 256))
    new_pol = {"vertices": coords, "RGBA": rgba}
    idx = np.random.randint(0, len(mutant) + 1)
    mutant.insert(idx, new_pol)


def mutation(original, size, num_verts=3, mutation_ways=("modify", "delete", "insert")):
    """



    :
    - modify:
                                                        ;
    - delete:
                                ;
    - insert:
                                .
    """
    mutant = copy.deepcopy(original)
    if len(mutant) == 0:
        new_pol = {"vertices": [(0, 0), (size[0], 0), (0, size[1])],
                   "RGBA": (128, 128, 128, 100)}
        mutant.append(new_pol)
        return mutant
    way = np.random.choice(mutation_ways)
    if way == "modify":
        mutation_modify(mutant, size)
    elif way == "delete":
```

```
        mutation_delete(mutant)
    elif way == "insert":
        mutation_insert(mutant, size, num_verts)
    return mutant


def pop_fitness(pop, target):
    """
                                        """
    size = target.size
    maxdiff = max_pixel_difference(target)
    fitvec = []
    for org in pop:
        img = draw(org[0], size)
        diff = pixel_difference(img, target)
        fitness = (1 - diff / maxdiff) * 100
        fitvec.append(fitness)
    return fitvec


def individ_fitness(individ, target):
    """
                        """
    size = target.size
    maxdiff = max_pixel_difference(target)
    img = draw(individ, size)
    diff = pixel_difference(img, target)
    return (1 - diff / maxdiff) * 100


def random_polygons(N, size, num_verts=3, colour="random", alpha="random"):
    """
                                        """
    collection = []
    for i in range(N):
        coords = [(np.random.randint(-10, size[0] + 10),
                   np.random.randint(-10, size[1] + 10)) for _ in range(num_verts)]
        if alpha == "random":
            a = np.random.randint(0, 50)
        else:
            a = alpha
        if colour == "white":
            rgba = (255, 255, 255, a)
```

```
        elif colour == "black":
            rgba = (0, 0, 0, a)
        elif colour == "random":
            rgba = (np.random.randint(0, 256), np.random.randint(0, 256),
                    np.random.randint(0, 256), a)
        triangle = {"vertices": coords, "RGBA": rgba}
        collection.append(triangle)
    return collection


def create_pop(Npop, N, size, target, num_verts=3, colour_init="black", alpha_init=100):
    """
                    .
                                        N
                                                                    .
    """
    pop = []
    for i in range(Npop):
        individ = random_polygons(
            N, size, num_verts=num_verts, colour=colour_init, alpha=alpha_init)
        fitness = individ_fitness(individ, target)
        pop.append((individ, fitness))
    pop = sorted(pop, key=lambda x: x[1], reverse=True)
    return pop


def selection_tournament(pop, fitness, k=3):
    """
                    """
    n = len(pop)
    indices = np.random.choice(n, size=min(k, n), replace=False)
    best_idx = indices[0]
    for i in indices[1:]:
        if fitness[i] > fitness[best_idx]:
            best_idx = i
    return best_idx


def crossover(parent1, parent2, size, pmut=0.1, num_verts=3):
    """
```

```
                                                    (
                                    ).
    """
    n = min(len(parent1), len(parent2))
    if n <= 1:
        child = copy.deepcopy(parent1 if np.random.random() < 0.5 else parent2)
    else:
        point = np.random.randint(1, n)
        child = copy.deepcopy(parent1[:point]) + copy.deepcopy(parent2[point:])
    if np.random.random() < pmut:
        child = mutation(child, size, num_verts=num_verts)
    return child


def new_generation(pop, fitness, size, target, pmut=0.1, num_of_olds=0.5, num_verts=3):
    """
                                        :                num_of_olds
                                ,                                    .
    """
    n_pop = len(pop)
    offspring = []
    for _ in range(n_pop):
        i1 = selection_tournament(pop, fitness)
        i2 = selection_tournament(pop, fitness)
        parent1 = pop[i1][0]
        parent2 = pop[i2][0]
        child = crossover(parent1, parent2, size, pmut=pmut, num_verts=num_verts)
        offspring.append(child)
    n_keep = max(1, int(n_pop * num_of_olds))
    n_need = n_pop - n_keep
    newpop = [pop[i][0] for i in range(n_keep)] + offspring[:n_need]
    fitness_new = [individ_fitness(ind, target) for ind in newpop]
    newpop = list(zip(newpop, fitness_new))
    newpop = sorted(newpop, key=lambda x: x[1], reverse=True)
    return newpop


def mean(pop):
    s = 0
    for elem in pop:
```

```
        s += elem[1]
    return s / len(pop)


def genetic_triangle_painting(target, Npop, Ntri=100, num_verts=3, maxgen=10, every=20,
                              pmut=0.1, num_of_olds=0.1, colour_init="random", alpha_init="random",
                              logs=True, outdir="output_images"):
    size = target.size
    os.makedirs(outdir, exist_ok=True)
    pop_time = time.time()
    pop = create_pop(Npop=Npop, N=Ntri, size=size, target=target, num_verts=num_verts,
                     colour_init=colour_init, alpha_init=alpha_init)
    fitness = pop_fitness(pop, target)
    pop_time = time.time() - pop_time
    info = []
    for i in range(maxgen + 1):
        maxfit = pop[0][1]
        if i % every == 0:
            avgfit = mean(pop)
            best = pop[0][0]
            best_img = draw(best, size=size)
            outpath = os.path.join(outdir, f"generation_{i}.png")
            best_img.save(outpath)
            info.append([i, maxfit, avgfit, pop_time, outpath])
            if logs:
                print(f"Generation: {i}    Max. Fitness: {np.round(maxfit, 2)}%    "
                      f"Avg. Fitness: {np.round(avgfit, 2)}%   Time: {pop_time}")
        pop_time = time.time()
        newpop = new_generation(pop, fitness, size, target, pmut=pmut, num_of_olds=num_of_olds,
                                num_verts=num_verts)
        fitness = pop_fitness(newpop, target)
        pop_time = time.time() - pop_time
        pop = newpop
    dfout = pd.DataFrame(
        info, columns=["generation", "max_fitness", "avg_fitness", "time", "outpath"])
    return dfout

```

```python

if __name__ == "__main__":
    image_path = "antony.jpg"
    target = Image.open(image_path).convert("RGB")
    df = genetic_triangle_painting(
        target,
        Npop=80,
        Ntri=300,
        num_verts=3,
        maxgen=400,
        every=20,
        pmut=0.15,
        num_of_olds=0.25,
        outdir="output_images"
    )
    print(df)
```

Результат представлен на рисунках 1– **??**.

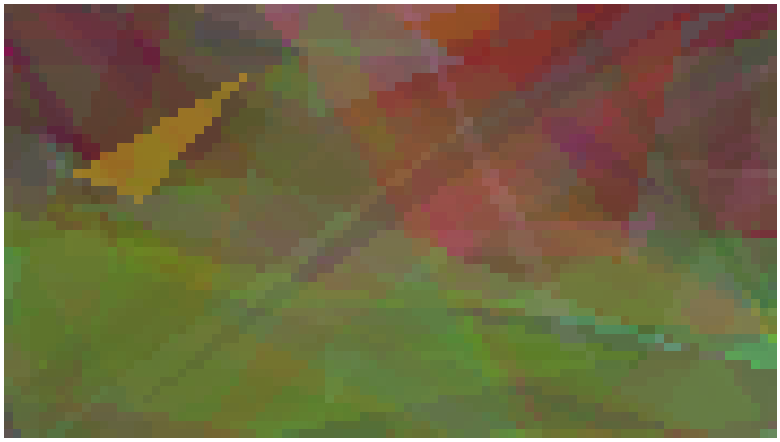

Рис. 1 — Результат